

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



LẬP TRÌNH NHÂN LINUX PHẦN 2. TỔNG QUAN

Ngành: Công nghệ thông tin

Chuyên ngành: Kỹ thuật phần
mềm nhúng và di động

Mã số: 52.48.02.01

Nội dung

- Introduction
- VFS core data structure
- Deeply inside file operation

Introduction

- File system là gì?
 - Là một hệ thống tổ chức, lưu trữ file của người dùng.
 - ✓ FAT, FAT32, NTFS, EXT4, network file system.
- Virtual file system
 - Là một lớp abstraction chung của các file system. Nhằm tạo ra một hệ thống file duy nhất cho hệ thống.

Các service cung cấp bởi VFS

- Tạo ra một hệ thống đọc/ghi file đồng nhất trong cả hệ thống
- Cung cấp các Pseudo file system
 - /proc, /sys, /tmp
- Hỗ trợ network file system
- Thao tác đọc ghi file giống nhau, bất kể file đang được lưu trữ ở đâu.

Các service cung cấp bởi VFS

- Cung cấp giao diện file cho người dùng.
- Mở file, đóng file,...
- Quản lý file permission, quyền sở hữu.
- Cached meta data của file
 - dùng lệnh grep lần 2 trong cùng 1 folder sẽ nhanh hơn lần 1
- Source code: `kernel_source/fs`

VFS Architecture

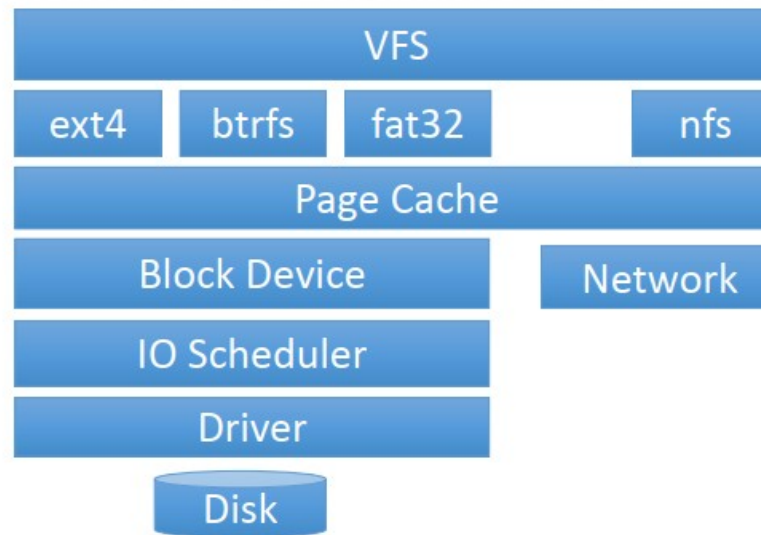
Fall 2014:: CSE 506:: Section 2 (PhD)



More detailed diagram

User

Kernel



File object

- Chứa các thông tin để mô tả phương thức để process tương tác với file.
- Được tạo ra khi process open 1 file.
- Không được lưu trữ sẵn trong ổ cứng
- `kernel_source/include/linux/fs.h`

File object

```
852 struct file {
853     union {
854         struct llist_node    fu_llist;
855         struct rcu_head      fu_rcuhead;
856     } f_u;
857     struct path              f_path;
858     struct inode              *f_inode;    /* cached value */
859     const struct file_operations *f_op;
860
861     /*
862      * Protects f_ep_links, f_flags.
863      * Must not be taken from IRQ context.
864      */
865     spinlock_t               f_lock;
866     enum rw_hint              f_write_hint;
867     atomic_long_t             f_count;
868     unsigned int              f_flags;
869     fmode_t                   f_mode;
870     struct mutex               f_pos_lock;
871     loff_t                     f_pos;
872     struct fown_struct         f_owner;
873     const struct cred          *f_cred;
874     struct file_ra_state       f_ra;
875
876     u64                        f_version;
877 #ifdef CONFIG_SECURITY
878     void                       *f_security;
```


Dentry object

- Mô tả cấu trúc các file trong thư mục
- Được tạo ra một loạt mỗi khi process mở 1 file
 - Được free khi không có bất cứ file nào trong thực đang được mở.
- Không lưu trữ sẵn trong ổ cứng.
- `kernel_source/include/linux/dcache.h`

Dentry object

```
83 struct dentry {
84     /* RCU lookup touched fields */
85     unsigned int d_flags;           /* protected by d_lock */
86     seqcount_t d_seq;              /* per dentry seqlock */
87     struct hlist_bl_node d_hash;   /* lookup hash list */
88     struct dentry *d_parent;       /* parent directory */
89     struct qstr d_name;
90     struct inode *d_inode;         /* Where the name belongs to - NULL is
91                                     * negative */
92     unsigned char d_iname[DNAME_INLINE_LEN]; /* small names */
93
94     /* Ref lookup also touches following */
95     struct lockref d_lockref;      /* per-dentry lock and refcount */
96     const struct dentry_operations *d_op;
97     struct super_block *d_sb;      /* The root of the dentry tree */
98     unsigned long d_time;          /* used by d_revalidate */
99     void *d_fsdata;               /* fs-specific data */
100
101     union {
102         struct list_head d_lru;    /* LRU list */
103         wait_queue_head_t *d_wait; /* in-lookup ones only */
104     };
105     struct list_head d_child;      /* child of parent list */
106     struct list_head d_subdirs;    /* our children */
107     /*
```

Inode object

- Chứa các thông dùng để quản lý file trong OS.
- Định danh bởi 1 số riêng biệt trong OS. Số này được tạo ra bởi OS và không lưu trên ổ cứng.
- `kernel_source/include/linux/fs.h`

Inode object

```
struct inode {
    umode_t          i_mode;
    unsigned short   i_opflags;
    kuid_t           i_uid;
    kgid_t           i_gid;
    unsigned int      i_flags;

#ifdef CONFIG_FS_POSIX_ACL
    struct posix_acl  *i_acl;
    struct posix_acl  *i_default_acl;
#endif

    const struct inode_operations *i_op;
    struct super_block *i_sb;
    struct address_space *i_mapping;

#ifdef CONFIG_SECURITY
    void *i_security;
#endif
}
```

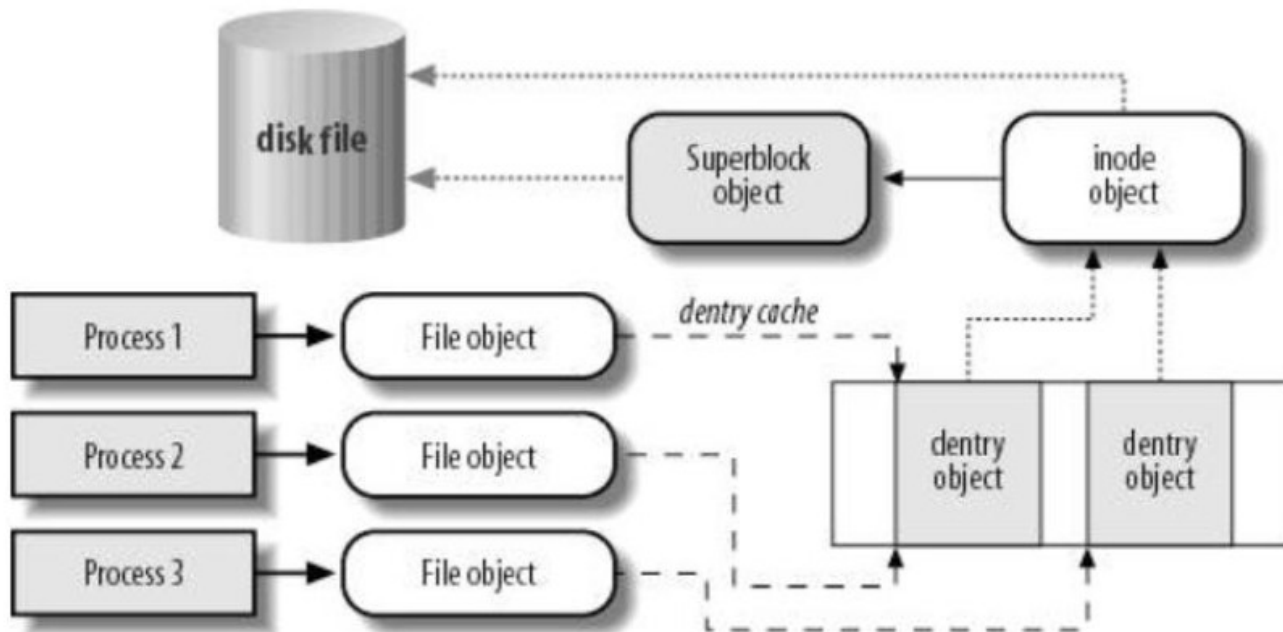
Super block object

- Chứa các thông tin chung của loại file system đó.
- FAT
 - Block size 512 byte
 - Max file size 4G
 - Không hỗ trợ recover data khi ghi file bị lỗi

Super block object

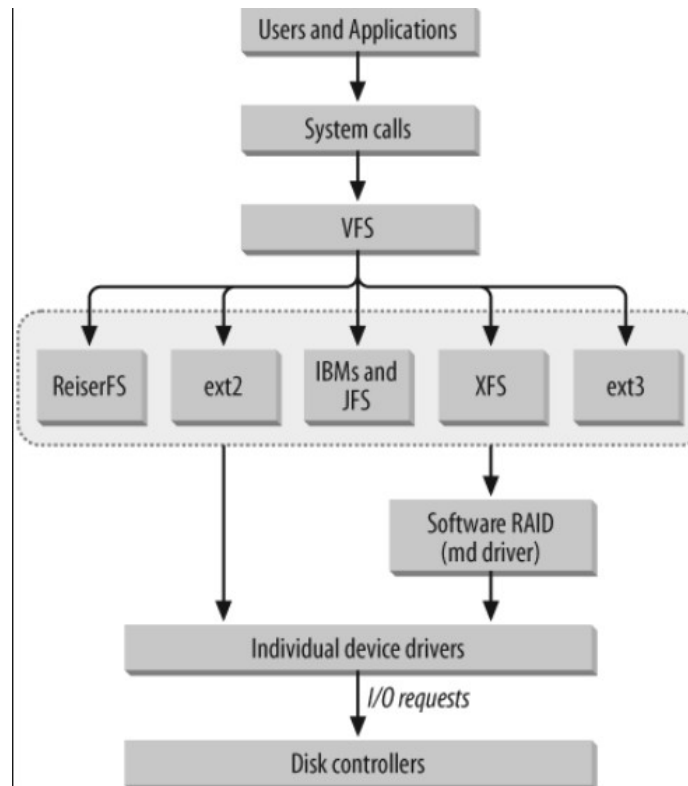
```
struct super_block {  
    struct list_head      s_list;          /* Keep this first */  
    dev_t                  s_dev;           /* search index; _not  
    unsigned char          s_blocksize_bits;  
    unsigned long          s_blocksize;  
    loff_t                 s_maxbytes;      /* Max file size */  
    struct file_system_type *s_type;  
    const struct super_operations *s_op;  
    const struct dquot_operations *dq_op;  
    const struct quotactl_ops *s_qcop;  
    const struct export_operations *s_export_op;  
    unsigned long          s_flags;  
    unsigned long          s_iflags;        /* internal SB_I_* fl  
    unsigned long          s_magic;
```

Relationship between VFS core data



From *Understanding Linux kernel, 3rd Ed*

File operation flow



Bài tập

- Sử dụng kernel module để tạo ra 1 file system. Kernel module tạo ra device file và cho phép user mount device file vào 1 folder khác. Sau khi mount user có thể nhìn vào mount folder và xem file trong đó.
 - Tham khảo: <https://kukuruku.co/post/writing-a-file-system-in-linux-kernel/>

HỎI - ĐÁP
